

Operating Systems Processes

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Source materials: this presentation includes topics discussed in Chapter 3 of the textbook, and it is an adaptation of the original book slides ch3.pptx.

Goals:

- Identify the components of a process
- Describe how processes are created and terminated
- Introduce design choices for interprocess communication

- **Multi-program:** more than one program in concurrent execution
- **Multi user:** more than one use can work on the computer
- **Time-sharing:** small time-slices shared across running programs
- **Process** is the abstraction to make it possible

- Program is **passive** entity stored on disk (executable file); process is **active**
 - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
 - Consider multiple users executing the same program

- Process is identified with an integer number

`pid_t pid;`

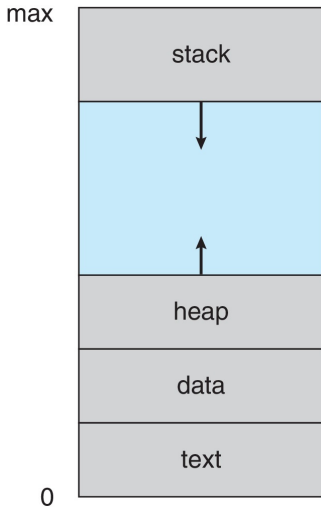
- Two processes must have different pids.
- List processes

`ps -ax`

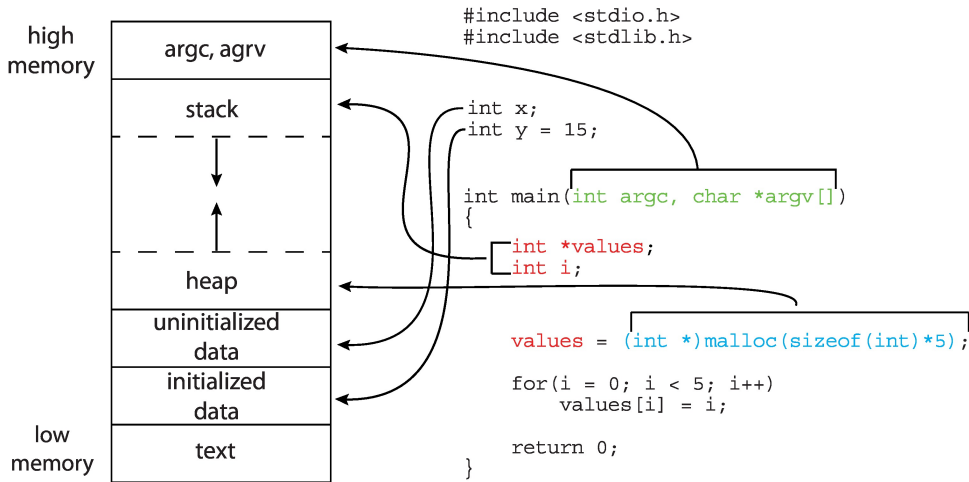
- `-a`: list all processes
- `-x`: list also processes not attached to terminals

- An operating system executes a variety of programs that run as a process
- **Process**: a program in execution
 - Process execution must progress in sequential fashion
 - No parallel execution of instructions of a single process
- Multiple parts
 - The program code, also called **text section**
 - Current activity including **program counter**, processor registers
 - **Stack** containing temporary data (function parameters, return addresses, local variables)
 - **Data section** containing global variables
 - **Heap** containing memory dynamically allocated during run time

Process in Memory



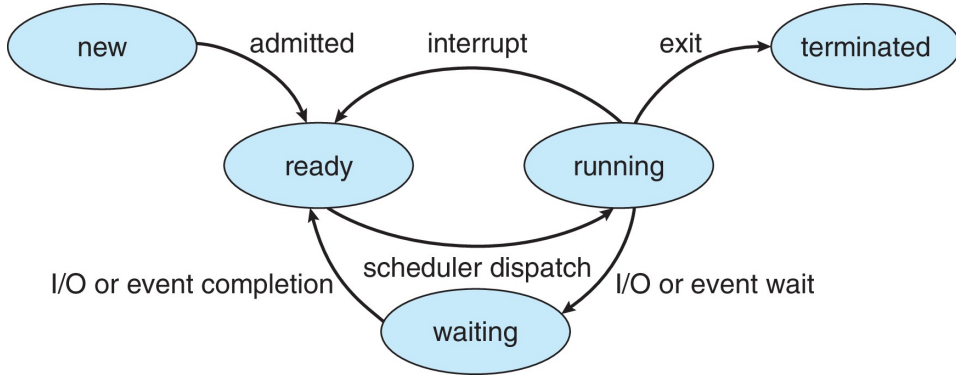
Memory Layout of a C Program



As a process executes, it changes **state**

- **New**: The process is being created
- **Running**: Instructions are being executed
- **Waiting**: The process is waiting for some event to occur
- **Ready**: The process is waiting to be assigned to a processor
- **Terminated**: The process has finished execution

Diagram of Process State



In Linux other states are present

- `TASK_INTERRUPTIBLE`: Task is waiting and the waiting can be stopped by a signal (a primitive of IPC)
- `TASK_UNINTERRUPTIBLE`: Task is waiting and the waiting cannot be stopped by a signal
- `EXIT_ZOMBIE`: The process is terminated, but its resources have not yet been deallocated. The process creator can still access its return state
- `EXIT_DEAD`: The process is terminated and the resources have been deallocated
- `TASK_STOPPED`: The task has been stopped by a terminal action (Ctrl+Z)
- `TASK_TRACED`: The task is traced by another process (e.g. a debugger) for step-by-step execution

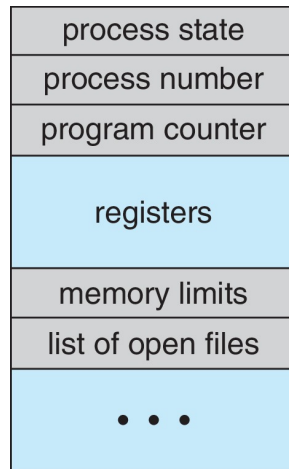
From `/include/linux/sched.h`

```
/* Used in tsk->__state: */
#define TASK_RUNNING 0x00000000
#define TASK_INTERRUPTIBLE 0x00000001
#define TASK_UNINTERRUPTIBLE 0x00000002
#define __TASK_STOPPED 0x00000004
#define __TASK_TRACED 0x00000008
#define EXIT_DEAD 0x00000010
#define EXIT_ZOMBIE 0x00000020
#define EXIT_TRACE (EXIT_ZOMBIE | EXIT_DEAD)
/* Used in tsk->__state again: */
#define TASK_PARKED 0x00000040
#define TASK_DEAD 0x00000080
#define TASK_WAKEKILL 0x00000100
#define TASK_WAKING 0x00000200
#define TASK_NOLOAD 0x00000400
#define TASK_NEW 0x00000800
/*Continue...-*/
```

Process Control Block (PCB)

Information associated with each process (also called task control block)

- **Process state** – running, waiting, etc.
- **Program counter** – address of next instruction to execute
- **CPU registers** – contents of registers for process
- **Scheduling info** – priorities, queue
- **Memory info** – memory allocated to the process
- **Accounting info** – CPU used, clock time elapsed since start, time limits
- **I/O info** – I/O devices allocated to process, list of open files



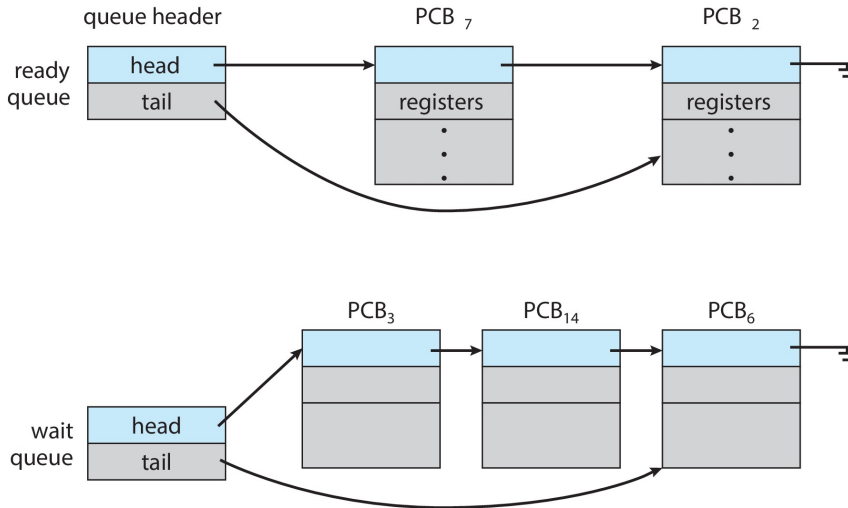
- So far, process has a single thread of execution
- Multiple execution flows per process → multiple info for each thread
 - **Per-thread** program counter
 - **Per-thread** registers
 - **Per-thread** stack

Shared info among thread

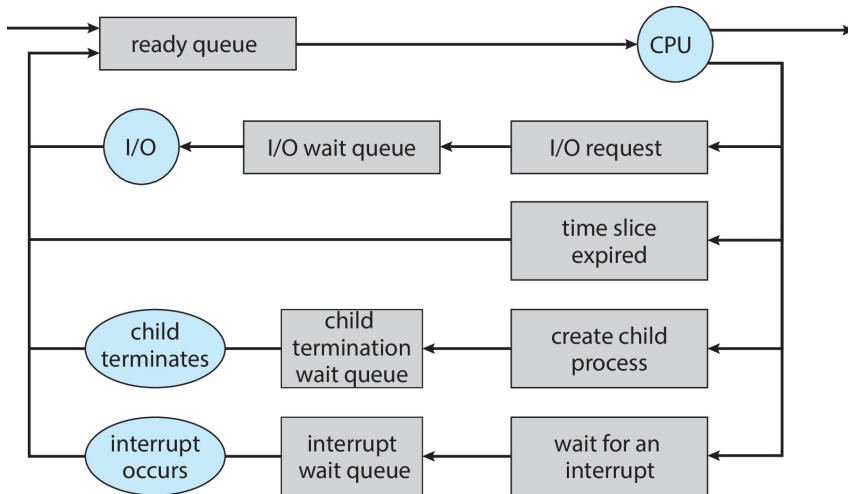
- **Per-process** memory info
 - **Per-process** open files
- Must store all per-thread info in PCB

- **Process scheduler** selects among available processes for next execution on CPU core
- Goal – Maximize CPU use, quickly switch processes onto CPU core
- Maintains scheduling queues of processes
 - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - **Wait queues** – set of processes waiting for an event (i.e., I/O)
 - Processes migrate among the various queues

Ready and Wait Queues

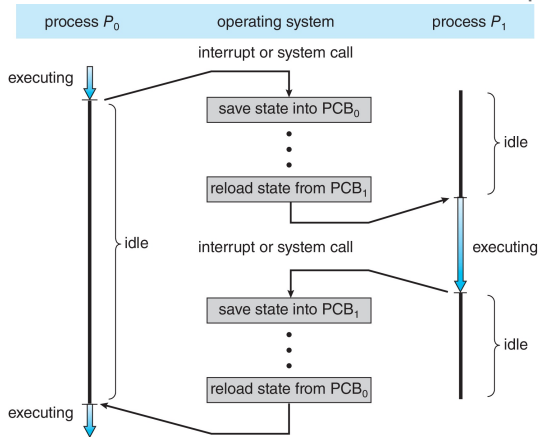


Representation of Process Scheduling



CPU Switch From Process to Process

A context switch occurs when the CPU switches from one process to another

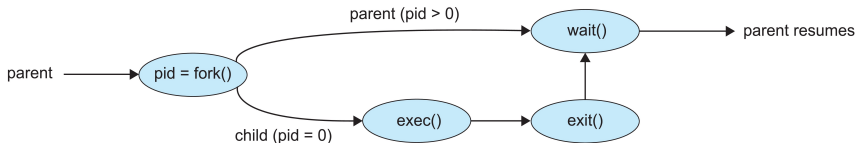


- When CPU switches to another process, the system must **save the state** of the old process and load the saved state for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is **pure overhead**; the system does no useful work while switching
 - The more complex the OS and the PCB → the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU → multiple contexts loaded at once

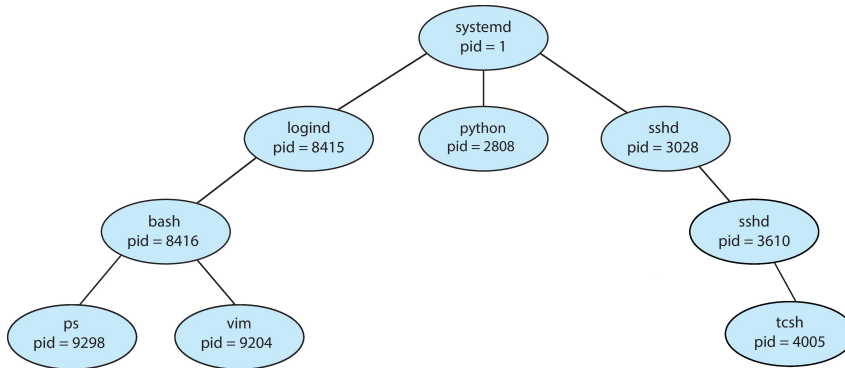
- System must provide mechanisms for
 - Process **creation**
 - Process **termination**

- **Parent** process create **children** processes, which, in turn create other processes, forming a tree of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - `fork()` system call creates new process
 - `exec()` system call used after a `fork()` to replace the process' memory space with a new program
 - Parent process calls `wait()` waiting for the child to terminate



A Tree of Processes in Linux



Esecution of `fork()` system call

- Creation of new process (different **pid** w.r.t. parent)
- Copy resources from parent to child
 - Memory (*Text*, *Heap*, *Stack* segments)
 - Open files (including Inter Process Communication (IPC) endpoints)
- Both child and parent resume execution from the same instruction
- Only difference: return code of `fork()` system call

Esecution of `exec()` system call

- Wipes data structures
 - Memory
 - Open files
- Load new program in memory (`mmap()`)
- Executes new program from the beginning

Considerations

- `fork()+exec()` highly inefficient
- When `fork()` is followed by `exec()` several operations can be omitted
- Current approach use of more flexible `clone()` system call

- `exec`**l**() – arguments are passed as a **list** of parameters terminated by a NULL pointer
- `exec`**v**() – arguments are passed as an array (**vector**) of pointers
- `exec`**le**(), `execv`**e**() – **environment** is passed as an array of pointers
- `exec`**lp**(), `execvp(), execvpe() – uses the PATH environment variable to find the executable file. If there is no p variant, the full path of the executable file must be specified`

- Parent process can wait for termination of a child process by using the `wait()` system call
- The call returns status information and the `pid` of the terminated process
- `wait()` system call can be used to wait for any child process to terminate

```
pid = wait(&status);
```

- `waitpid()` system call can be used to wait for a specific child process to terminate

```
pid = waitpid(child_pid, &status, options);
```

- If parent does not invoke `wait()` the child process becomes a **zombie** when it terminates
 - Process is terminated, but its resources have not yet been deallocated
 - The process creator can still access its return state
- If parent terminated without invoking `wait()`, the child process becomes an **orphan**
 - In Linux, orphans are **re-parented** by the **init** process, that **reaps** (as in the grim reaper) them

C Program Forking Separate Process

```
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>
int main () {
    pid_t pid; /* fork a child process */
    pid = fork ();
    if (pid < 0) { /* error occurred */
        fprintf (stderr, "Fork Failed\n");
        return 1;
    } else if (pid == 0) { /* child process */
        execlp ("/bin/ls", "ls", NULL);
    } else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf ("Child Complete\n");
    }
}
```

- Process executes last statement and then asks the operating system to delete it using the `exit()` system call.
 - Returns status data from child to parent (via `wait()`)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the `abort()` system call. Some reasons for doing so
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates

Fork bomb in C:

```
while (1){fork();}
```

